

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. MODULAR COMPONENT ARCHITECTURE:

To guarantee long-term maintainability and rapid debugging, the ComicCraft application is built upon a strict Modular Component Architecture. The repository is logically divided into specific directories separating backend logic, frontend views, and static assets:

- `app/`: The central directory containing all Python backend operations.
 - `main.py`: Initializes the application and configures the FastAPI web server.
 - `app_routes.py`: The primary router that intercepts HTTP requests and dictates the sequence of the AI generation pipeline.
 - `gemini_logic.py`: A dedicated script to manage HTTPS payload construction and parsing for the Google Generative AI endpoints.
 - `image_synthesis.py`: Manages network calls to Hugging Face, specifically equipped with built-in network timeout fallbacks.
 - `pdf_engine.py`: Encapsulates the FPDF logic, calculating pixel coordinates to align downloaded images with AI-generated text.
- `templates/`: Contains all Jinja2 HTML views (`index.html`, `preview.html`) used to display data dynamically to the end-user.
- `static/`: The designated storage for CSS files, background graphics, dynamically generated `.jpg` panels (`static/panels/`), and the finalized `.pdf` exports (`static/exports/`).

3. CODE READABILITY STANDARDS

- **Strict Formatting:** All Python modules strictly adhere to PEP 8 standards. Complex operations, such as PDF coordinate mathematics and network exception handling, feature comprehensive inline documentation to assist future developers.

- **Descriptive Naming:** Variable and function names are highly explicit (e.g., `scene_blueprint`, `synthesize_image`, `fetch_outline_from_gemini`), avoiding ambiguous abbreviations and ensuring the logic is instantly understandable.
- **Separation of Concerns:** No single file exceeds its intended purpose. The routing logic never handles image saving, and the LLM module never touches the PDF generation, ensuring a clean separation of concerns.

4. REUSABILITY & SCALABILITY

- **Agnostic Prompt Injection:** The AI handler scripts (`gemini_logic.py` and `image_synthesis.py`) are designed to accept highly dynamic string variables. This allows the exact same functions to generate a Sci-Fi story or a Fantasy story without any structural code changes.
- **Flexible Loop Iteration:** The PDF engine uses dynamic `for`-loops to calculate document length. Whether the AI returns 3 panels or 10 panels, the compiler automatically adjusts the page count without requiring manual code updates.
- **Environment Isolation:** Sensitive credentials are removed from the source code and stored in a `.env` file. This means the codebase can be cloned, shared, and reused by other developers simply by injecting their own API keys.